Security Audit Report

Valdora Vault Contract

v1.0

April 17, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 7
Methodology 8
Functionality Overview 8
How to Read This Report 9
Code Quality Criteria 10
Summary of Findings 11
Detailed Findings 13
1. Redemption rejection consumes minting capacity and does not restore TOTAL_BURNED, breaking supply tracking and future mints 13
2. Pending redemptions inflate NAV because shares are burned before AUM is reduced 14
3. Approving a redemption request without override_nav can silently zero LATEST_AUM, wiping NAV for all remaining shareholders 14
4. AUM change limit bypass via repeated small updates 15
5. Pending admin transfer is not cleared on migration 15
6. Pause state is not enforced on privileged operations that affect share pricing 16
7. Unbounded NAV override in redemption approval 16
8. Users can burn shares for zero asset return due to integer rounding 17
9. Unpaginated redemption request and role queries 17
10. Deposit event's total_supply attribute emits gross cumulative minted counter 18
11. migrate does not verify the previous contract name or version 18
12. Untracked assets sent directly to an InternallyManaged vault are stranded 19
13. No redeem_manager role granted during instantiation for externally managed vaults 19
14. Role and admin state operations emit events without idempotency checks 20
15. Missing idempotency checks in pause and unpause 20
16. Role names not validated against known constants 21
17. Misleading error messages in payment validation 21
18. Redundant storage loads and external queries in the deposit flow 22
19. Instantiation does not enforce the separation of privileged roles 22
20. NAV representation as an integer loses sub-unit precision 23
21. Supply tracking diverges if shares are burned outside the vault 23
22. Contract-side minting cap check diverges from chain-side validation 24
23. Unreachable unwrap_or fallbacks on guarded arithmetic branches 24

24. Unreachable zero-amount checks after must_pay validation 25
25. Identical RedemptionRequest field naming reduces code readability 25
26. FUNDS_MANAGER_ADDRESS is set unconditionally regardless of vault type 26
27. Misleading doc comment describes unsupported behavior for InternallyManaged 26
28. Duplicate CONTRACT_NAME and CONTRACT_VERSION constants are defined 27

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Highend Technologies LLC to perform a security audit of the Valdora Vault Contract.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/Liquid-Zig/vault-contracts-audit
Commit	`57c3f00b63fc06a897a091768b378bc33c868a29`
Scope	All files were in scope, excluding the following: • `packages/` — ZigChain bindings and macros (chain-level primitives, not vault logic) • `contracts/vault/src/multitest/` — test harness and test cases • `e2e/` — TypeScript deployment and query scripts • `analytics-indexer/` — off-chain indexer • `deployment/` — deployment configs • Build scripts, documentation, CI
Fixes verified at commit	`b5c583f8345339cae2b380e264995bfd100ca7cd` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Valdora Vault is a CosmWasm smart contract on ZigChain that implements a NAV-based vault where users deposit a single asset (e.g., USDC) and receive vault shares (a native ZigChain factory token). The vault supports two modes: `ExternallyManaged` (funds forwarded to a funds manager, oracle-driven AUM, queued redemptions) and `InternallyManaged` (funds held in contract, reward-based AUM growth, instant redemptions).

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**Low-Medium**	-
Code readability and clarity	**Medium**	-
Level of documentation	**Medium**	-
Test coverage	**Medium-High**	`cargo tarpaulin` reports a coverage of `89.24%`.

Summary of Findings

No	Description	Severity	Status
1	Redemption rejection consumes minting capacity and does not restore `TOTAL_BURNED`, breaking supply tracking and future mints	Critical	Resolved
2	Pending redemptions inflate NAV because shares are burned before AUM is reduced	Critical	Resolved
3	Approving a redemption request without `override_nav` can silently zero `LATEST_AUM`, wiping NAV for all remaining shareholders	Major	Resolved
4	AUM change limit bypass via repeated small updates	Minor	Resolved
5	Pending admin transfer is not cleared on migration	Minor	Resolved
6	Pause state is not enforced on privileged operations that affect share pricing	Minor	Resolved
7	Unbounded NAV override in redemption approval	Minor	Resolved
8	Users can burn shares for zero asset return due to integer rounding	Minor	Resolved
9	Unpaginated redemption request and role queries	Minor	Resolved
10	Deposit event's `total_supply` attribute emits gross cumulative minted counter	Informational	Resolved
11	`migrate` does not verify the previous contract name or version	Informational	Resolved
12	Untracked assets sent directly to an `InternallyManaged` vault are stranded	Informational	Resolved
13	No `redeem_manager` role granted during instantiation for externally managed vaults	Informational	Resolved
14	Role and admin state operations emit events without idempotency checks	Informational	Resolved
15	Missing idempotency checks in pause and unpause	Informational	Resolved
16	Role names not validated against known constants	Informational	Resolved

17	Misleading error messages in payment validation	**Informational**	**Resolved**
18	Redundant storage loads and external queries in the deposit flow	**Informational**	**Resolved**
19	Instantiation does not enforce the separation of privileged roles	**Informational**	**Acknowledged**
20	NAV representation as an integer loses sub-unit precision	**Informational**	**Acknowledged**
21	Supply tracking diverges if shares are burned outside the vault	**Informational**	**Resolved**
22	Contract-side minting cap check diverges from chain-side validation	**Informational**	**Resolved**
23	Unreachable `unwrap_or` fallbacks on guarded arithmetic branches	**Informational**	**Resolved**
24	Unreachable zero-amount checks after `must_pay` validation	**Informational**	**Resolved**
25	Identical `RedemptionRequest` field naming reduces code readability	**Informational**	**Resolved**
26	`FUNDS_MANAGER_ADDRESS` is set unconditionally regardless of vault type	**Informational**	**Resolved**
27	Misleading doc comment describes unsupported behavior for `InternallyManaged`	**Informational**	**Resolved**
28	Duplicate `CONTRACT_NAME` and `CONTRACT_VERSION` constants are defined	**Informational**	**Resolved**

Detailed Findings

1. Redemption rejection consumes minting capacity and does not restore `TOTAL_BURNED`, breaking supply tracking and future mints

Severity: Critical

In `contracts/vault/src/contract/exec.rs:195-203`, `execute_request_redeem` burns shares via `BurnTokens` and increments `TOTAL_BURNED`. On the ZigChain `x/factory` side, the `Minted` counter is cumulative and never decremented by burns. The effective minting limit is computed as `maxSupply = MintingCap - totalBurned`, so each burn permanently consumes minting capacity.

However, in `exec.rs:404-409`, `execute_reject_request` remints the user's shares via `MintAndSendTokens`. The issue is that this remint increments the chain's cumulative `Minted` counter again, consuming additional minting capacity. The vault also does not decrement `TOTAL_BURNED`, so `query_total_supply` (which computes `minted - TOTAL_BURNED`) returns a value lower than the actual circulating supply.

Consequently, minting capacity is permanently consumed. Every burn reduces `maxSupply` on-chain, and a subsequent re-mint adds to `Minted` without restoring `maxSupply`. After enough burn/re-mint cycles, `Minted` will exceed `maxSupply`, causing all future mints (i.e., deposits and rejections) to fail on-chain.

Additionally, `TOTAL_BURNED` is incremented on burn but never decremented on re-mint, so the vault undercounts the circulating supply. This inflates the NAV and causes new depositors to receive fewer shares than they would at fair value.

Recommendation

We recommend implementing the following changes in `execute_reject_request`:

1. Before dispatching `MintAndSendTokens`, call `UpdateDenomMintingCap` to increase the cap by the re-minted amount (e.g., `denom_info.minting_cap.checked_add(request.shares)`) to ensure the chain does not reject the mint.
2. Decrement `TOTAL_BURNED` by `request.shares` to restore correct supply tracking.

Status: Resolved

2. Pending redemptions inflate NAV because shares are burned before AUM is reduced

Severity: Critical

In `ExternallyManaged` vaults, `execute_request_redeem` in `contracts/vault/src/contract/exec.rs:196-203` burns shares and increments `TOTAL_BURNED` but does not debit `LATEST_AUM`. AUM is only decremented on `execute_approve_request` in lines `345-351`.

During this window, NAV (`AUM / total_supply`) is inflated because `total_supply` drops while AUM remains unchanged. Any deposit during this window silently receives fewer shares than it would at fair value, and the shortfall flows to existing holders.

Consequently, any users who deposit during this window will receive fewer shares per unit of deposited assets. For example, with AUM = 1,000,000 and supply = 1,000,000 (NAV = 1), a redemption of 500,000 shares drops supply to 500,000 while AUM stays at 1,000,000, doubling NAV to 2. A new depositor of 100,000 assets receives only 50,000 shares instead of 100,000, suffering approximately 45% loss.

Recommendation

We recommend restoring the `total_supply` to `AUM` invariant in the `ExternallyManaged` branch, either by debiting `LATEST_AUM` by `asset_amount` in `execute_request_redeem` and re-crediting it in `execute_reject_request`, or by deferring both the burn and `TOTAL_BURNED` increment to `execute_approve_request`.

Status: Resolved

3. Approving a redemption request without `override_nav` can silently zero `LATEST_AUM`, wiping NAV for all remaining shareholders

Severity: Major

In `contracts/vault/src/contract/exec.rs:191`, `execute_request_redeem` computes `asset_amount` from the AUM and total supply at request creation time and stores it in the `RedemptionRequest`. Between request creation and approval, the oracle can legitimately reduce AUM via the `execute_update_aum` entry point in line `427`.

However, when the `redeem_manager` calls `execute_approve_request` without providing `override_nav`, the guard in line `346` compares the original `request.asset_amount` against the now-lower `current_aum`. If AUM falls below the locked amount, the `>=` branch fires, setting `LATEST_AUM` to zero even though other shareholders still hold shares.

The `redeem_manager` can avoid this by providing `override_nav` at `exec.rs:315-321`, which recalculates the `request.asset_amount` before the guard is reached. However, there is no enforcement or warning, as the contract silently zeros AUM instead of reverting when the original amount exceeds current AUM.

Consequently, if the `redeem_manager` approves without overriding, all remaining shareholders' NAVs drop to zero, and all future deposits are blocked by the `ZeroAum` error in `calculate_shares`.

Recommendation

We recommend reverting the approval if `request.asset_amount >= current_aum`. This forces the `redeem_manager` to explicitly use `override_nav` to set a payout that does not exceed the current AUM.

Status: Resolved

4. AUM change limit bypass via repeated small updates

Severity: Minor

In `contracts/vault/src/contract/exec.rs:446-468`, the `execute_update_aum` entry point enforces a per-transaction bps limit but has no cooldown, epoch restriction, or cumulative limit. Each update uses the current AUM as the reference base, so the limit compounds.

A compromised `aum_updater` key can manipulate NAV far beyond the intended safety bounds within a single block. For example, with a 200 bps (2%) default in `DEFAULT_AUM_CHANGE_LIMIT_BPS`, 35 consecutive calls could achieve an approximately 50% decrease or a 100% increase.

Recommendation

We recommend adding either a time-based cumulative limit, a minimum cooldown between AUM updates, or both.

Status: Resolved

5. Pending admin transfer is not cleared on migration

Severity: Minor

The `migrate` function in `contracts/vault/src/contract.rs:144-168` can set a new admin directly, but does not clear `PENDING_ADMIN`.

Consequently, if a two-step admin transfer is in progress when a migration overrides `ADMIN`, the previously nominated pending admin can still call `execute_accept_admin` in

`contracts/vault/src/contract/exec.rs:708-734`, overwriting the governance-set admin.

Recommendation

We recommend clearing `PENDING_ADMIN` to `None` in the `migrate` function whenever a new admin is set.

Status: Resolved

6. Pause state is not enforced on privileged operations that affect share pricing

Severity: Minor

In `contracts/vault/src/contract/exec.rs:73` and `175`, the `assert_not_paused` function is only implemented in the `execute_deposit` and `execute_request_redeem` entry points. Other entry points do not check the pause state.

This is problematic because privileged operations that directly affect share pricing, such as `execute_update_aum`, `execute_add_reward`, `execute_approve_request`, and `execute_reject_request`, all work normally while the contract is paused. This creates an asymmetric state in which the `aum_updater` can move NAV and the `redeem_manager` can settle requests, while users cannot deposit or redeem.

Consequently, pricing can shift during a pause window without any user being able to act on it.

Recommendation

We recommend extending `assert_not_paused` to all state-modifying operations that affect share pricing. At a minimum, `execute_update_aum` and `execute_add_reward` should check the pause state.

Alternatively, if privileged operations are intentionally permitted during a pause, we recommend explicitly documenting this behavior.

Status: Resolved

7. Unbounded NAV override in redemption approval

Severity: Minor

The `execute_approve_request` entry point in `contracts/vault/src/contract/exec.rs:291` allows the redeem manager to pass an arbitrary `override_nav` that recalculates `request.asset_amount` as `shares * nav` with no constraint on how far the override deviates from the original NAV recorded at

request time (`request.current_nav`). This is problematic because a compromised or mistaken redeem manager could set the override to an inflated value, draining vault funds beyond the user's fair share.

We classify this issue as minor because it can only be caused by the redeem manager (`ROLE_REDEEM_MANAGER`), which is a privileged role. The override path inherits that trust assumption, but no secondary guardrail limits the magnitude of the change.

Recommendation

We recommend enforcing a maximum deviation threshold (e.g., a configurable percentage of the original `request.current_nav`) and rejecting any `override_nav` that falls outside that band.

Status: Resolved

8. Users can burn shares for zero asset return due to integer rounding

Severity: Minor

In `contracts/vault/src/contract/exec.rs:188-192`, the asset amount for a redemption is computed as `shares.multiply_ratio(aum, total_supply)`. If `aum` is very small relative to `total_supply`, integer rounding truncates the result to zero.

Since there is no validation preventing a zero-amount redemption, users will essentially be redeeming shares for nothing. For `InternallyManaged` vaults, the user's shares are burned immediately, and no assets are returned, resulting in a direct loss. For `ExternallyManaged` vaults, a zero-amount request is queued with shares already burned, and if approved, returns nothing.

Recommendation

We recommend adding a check that `asset_amount` is non-zero before proceeding with the redemption.

Status: Resolved

9. Unpaginated redemption request and role queries

Severity: Minor

The `query_pending_requests` and `query_user_requests` functions in `contracts/vault/src/contract/query.rs:100` and `116` iterate over every entry in the `REDEMPTION_REQUESTS` map using `.range(deps.storage, None, None, Order::Ascending)` with no `start_after` or `limit` parameters.

This is problematic because, as entries accumulate, these queries consume increasing gas and may exceed the node's query gas limit, rendering them unusable.

A similar issue applies to the `query_role_members` function in line `144`, which performs an unbounded iteration over the `ROLES` state entries.

Recommendation

We recommend adding pagination parameters (`start_after: Option<u64>`, `limit: Option<u32>`) to these query functions and capping the maximum number of results returned per call.

Status: Resolved

10. Deposit event's `total_supply` attribute emits gross cumulative minted counter

Severity: Informational

In `src/contract/exec.rs:94`, `141`, and `157`, the deposit event's `total_supply` attribute emits `denom_info.minted + shares`, which is the gross cumulative mint counter (never decremented by burns), not the circulating supply.

Consequently, off-chain indexers and front-ends that consume this event will display inflated supply and misreport vault share economics.

Recommendation

We recommend emitting `denom_info.minted - TOTAL_BURNED + shares` instead.

Status: Resolved

11. `migrate` does not verify the previous contract name or version

Severity: Informational

In `src/contract.rs:144-168`, `migrate` unconditionally calls `set_contract_version(CONTRACT_NAME, CONTRACT_VERSION)` without loading the previously-stored cw2 metadata. It does not verify the previous `contract` name matches `CONTRACT_NAME`, does not compare semver (allowing silent downgrades), and provides no hook for state-shape migration.

A cross-contract migration would silently overwrite the target contract's cw2 record, and future schema changes have no anchor for translation logic.

Recommendation

We recommend implementing the standard cw2 pattern: call `cw2::get_contract_version`, verify the stored `contract` equals `CONTRACT_NAME`, reject downgrades, branch on previous version for any state translation, then call `set_contract_version`.

Status: Resolved

12. Untracked assets sent directly to an `InternallyManaged` vault are stranded

Severity: Informational

In `src/contract/exec.rs:498-547` and `170-289`, for `InternallyManaged` vaults, `LATEST_AUM` is only increased via `execute_deposit` or `execute_add_reward`. Any asset sent directly to the contract address via a plain `BankMsg::Send` increases the contract's bank balance but does not credit AUM.

Since NAV is computed from `LATEST_AUM` rather than from the contract's actual balance, such funds cannot be recovered and do not benefit existing shareholders.

Recommendation

We recommend either prominently documenting that direct transfers are unrecoverable or adding an admin-only sweep that credits any unaccounted-for bank balance to `LATEST_AUM`.

Status: Resolved

13. No `redeem_manager` role granted during instantiation for externally managed vaults

Severity: Informational

In `src/contract/init.rs:12-79`, `instantiate` grants the `aum_updater` role at deployment, but does not grant the `redeem_manager` role.

For `ExternallyManaged` vaults, users can deposit and submit redemption requests before the admin assigns the role. Their shares are burned immediately on request, but no one can approve or reject until the admin separately calls `GrantRole`, leaving requests stuck in `Pending`.

Recommendation

We recommend either accepting a `redeem_manager_address` in `InstantiateMsg` (similar to `aum_updater_address`) or clearly documenting that the admin must grant the `redeem_manager` role before the vault begins accepting redemption requests.

Status: Resolved

14. Role and admin state operations emit events without idempotency checks

Severity: Informational

The `execute_revoke_role` function in `contracts/vault/src/contract/exec.rs:572` calls `ROLES.remove` without checking whether the role was granted to the target address, succeeding and emitting a `role_revoked` event even when the address never held the role.

The same pattern applies to `execute_grant_role` in line `549`, which saves a role without checking whether it is already granted, and to `execute_revoke_admin_transfer` in line `743`, which sets `PENDING_ADMIN` to `None` without checking whether a transfer is pending.

In both cases, the transaction succeeds and emits a misleading event for a no-op state change.

Recommendation

We recommend loading the current state before mutation and returning an error if the state is already in the desired state.

Status: Resolved

15. Missing idempotency checks in pause and unpause

Severity: Informational

The `execute_pause` and `execute_unpause` functions in `contracts/vault/src/contract/exec.rs:595` and `613` unconditionally overwrite `PAUSED` without checking whether the contract is already in the target state. Calling `execute_pause` on an already-paused vault succeeds and emits a duplicate `vault_paused` event, and likewise for `execute_unpause`.

Consequently, this can mislead off-chain monitoring systems that count or react to these events, leading them to believe that multiple state transitions occurred.

Recommendation

We recommend loading `PAUSED` at the start of each function and returning an error if the contract is already in the requested state.

Status: Resolved

16. Role names not validated against known constants

Severity: Informational

The `execute_grant_role` function in `contracts/vault/src/contract/exec.rs:549-570` accepts any arbitrary string as the role parameter without validating against known role constants (i.e., `aum_updater`, `redeem_manager`, `add_reward`). This is problematic because a typo silently succeeds, granting a role without any code-path checks, while the intended role remains unassigned.

The same issue applies to `execute_revoke_role` in lines `572-593`.

Recommendation

We recommend validating the role parameter against a whitelist of known role names or using a `Role` enum as the map key.

Status: Resolved

17. Misleading error messages in payment validation

Severity: Informational

In `execute_deposit` in `contracts/vault/src/contract/exec.rs:75-78`, all `must_pay` errors are mapped to `VaultError::InvalidAssetDenom` with "no asset sent", regardless of the actual failure (e.g., wrong denom, multiple coins, zero amount).

The same issue occurs in `execute_add_reward` in lines `511-514`. If a user sends two denominations, the `MultipleDenoms` error is replaced with a misleading "no asset sent" message.

Recommendation

We recommend preserving the original `PaymentError` variant or matching on error variants to provide specific error messages.

Status: Resolved

18. Redundant storage loads and external queries in the deposit flow

Severity: Informational

In `contracts/vault/src/contract/exec.rs:68`, `execute_deposit` performs redundant storage reads and external module queries on every deposit. `LATEST_AUM` is loaded three times: once in `calculate_shares` at line `55`, once in `get_current_nav` at line `43`, and once directly at line `115`.

The ZigChain denom query `ZQuerier::denom()` is also called three times: twice via `query_total_supply` in `contracts/vault/src/contract/query.rs:25` and once directly at line `93`.

Recommendation

We recommend refactoring `execute_deposit` to load `LATEST_AUM`, `FULL_DENOM`, and `TOTAL_BURNED` once, query `ZQuerier::denom()` once, and pass the results as parameters to `calculate_shares` and `get_current_nav`.

Status: Resolved

19. Instantiation does not enforce the separation of privileged roles

Severity: Informational

The `instantiate` function in `contracts/vault/src/contract/init.rs:12` accepts `admin`, `aum_updater_address`, `funds_manager_address`, and `metadata_admin` as independent parameters but never verifies that they are distinct addresses.

Consequently, a deployer can set the same address for multiple or all privileged roles, violating the principle of separation of privileges. If the key is compromised, the attacker gains control of every privileged operation.

Recommendation

We recommend adding validation in `instantiate` that rejects configurations in which any two of `admin`, `aum_updater_address`, `funds_manager_address`, and `metadata_admin` share the same address, or, at a minimum, documenting the expected deployment configuration.

Status: Acknowledged

The client states that the roles will be held by single or multiple multisigs. This is documented in `README.md` under the “Key Concepts” section.

20. NAV representation as an integer loses sub-unit precision

Severity: Informational

In `contracts/vault/src/contract/exec.rs:316`, the override NAV is multiplied with shares using `request.shares.checked_mul(nav)`, where `nav` is a `Uint256`. Integer representation means that any NAV value less than one full unit is truncated to zero, and fractional differences between assets and shares are lost.

While the core share/asset calculations use `multiply_ratio`, which handles this correctly, the stored and event-emitted NAV values suffer from this precision loss. Using `Decimal256` for NAV representation would preserve subunit precision and prevent potential integer-division artifacts in downstream calculations.

Recommendation

We recommend representing NAV as `Decimal256`.

Status: Acknowledged

21. Supply tracking diverges if shares are burned outside the vault

Severity: Informational

In `contracts/vault/src/contract/query.rs:22-37`, the `query_total_supply` function computes circulating supply as `denom_info.minted - TOTAL_BURNED`. Since ZigChain's `BurnTokens` handler is permissionless, any account holding vault shares can burn them directly via the factory module, bypassing the vault.

If a user burns shares externally, `bankKeeper.GetSupply()` decreases, but the vault's `TOTAL_BURNED` stays unchanged. This causes the vault to overstate supply, to understate NAV, and to result in new depositors receiving more shares than their fair value warrants.

While it is unlikely that users would burn their share tokens, doing so would break the accounting invariant.

Recommendation

We recommend using a Stargate query to `bankKeeper.GetSupply()` for the true circulating supply instead of computing it from `minted - TOTAL_BURNED`.

Alternatively, consider documenting this assumption and accepting the risk.

Status: Resolved

22. Contract-side minting cap check diverges from chain-side validation

Severity: Informational

In `contracts/vault/src/contract/exec.rs:94-103`, the deposit flow checks whether `denom_info.minted + shares > denom_info.minting_cap` before dispatching the mint message.

However, on the ZigChain chain side, the actual validation computes `maxSupply = MintingCap - (Minted - bankSupply)`, which accounts for previously burned tokens.

Consequently, since `maxSupply = MintingCap - burned` is always less than or equal to `MintingCap`, the contract check is strictly weaker than the chain check. When tokens have been burned, a deposit can pass the contract's cap validation but then fail on-chain. The user will then receive an opaque SDK error instead of the descriptive `VaultError::ExceedsMintingCap` that the contract-level check is designed to provide.

Recommendation

We recommend aligning the contract-side check with the chain's `maxSupply` formula by accounting for burned supply, or removing the redundant pre-check and relying on the chain-side validation.

Status: Resolved

23. Unreachable `unwrap_or` fallbacks on guarded arithmetic branches

Severity: Informational

In `contracts/vault/src/contract/exec.rs:260`, the expression `aum.checked_sub(asset_amount).unwrap_or(Uint256::zero())` is executed when `asset_amount < aum`. This is problematic because it means that the `unwrap_or` fallback is not unreachable.

The same pattern appears in line `349` for another guarded subtraction, and in lines `479` and `530` for division operations where the preceding `if total_supply.is_zero()` branch already handles the zero-divisor case.

Consequently, these fallbacks silently mask potential errors. If a guard condition were ever changed or removed, the `unwrap_or` would suppress the failure by returning zero, which could propagate an incorrect value through subsequent calculations without any indication.

Recommendation

We recommend replacing `unwrap_or(Uint256::zero())` with a propagating error (e.g., using `?` or an explicit `map_err`) so that any unexpected arithmetic failure surfaces immediately rather than being silently absorbed.

Status: Resolved

24. Unreachable zero-amount checks after `must_pay` validation

Severity: Informational

In `contracts/vault/src/contract/exec.rs:80`, `if asset_amount.is_zero()` checks whether the deposit amount is zero. This check is unreachable because `must_pay` (from `cw-utils`) already returns an error when funds are zero or missing.

This issue also occurs in the following instances:

- Line `181` and `516`, where `shares.is_zero()` is checked after `must_pay` validates the share payment.
- In line `188`, `total_supply.is_zero()` is checked during a redemption. Since the user just attached shares via `must_pay`, at least those shares exist in the supply, making the zero-supply branch unreachable. If this branch were ever entered, the user would receive zero assets for their burned shares without any error.

Recommendation

We recommend removing the unreachable `is_zero()` checks.

Status: Resolved

25. Identical `RedemptionRequest` field naming reduces code readability

Severity: Informational

In `contracts/vault/src/contract/exec.rs:221`, a `RedemptionRequest` is created with both `approved_at` and `fulfilled_at` fields. In the current implementation, approval and fulfillment happen in the same transaction during `ApproveRequest`, so these two fields always hold the same value and serve no distinct purpose.

On the other hand, in line `319`, the `current_nav` field is overwritten with the override value when an `override_nav` is provided. After this assignment, `current_nav` and `override_nav` end up with the same value, and the original NAV at the time of the request is lost. A field name like `original_nav` would better convey the intent of preserving the NAV snapshot from when the request was created.

Recommendation

We recommend consolidating `approved_at` and `fulfilled_at` into a single timestamp field if they always coincide, or documenting when they would differ.

For the NAV fields, consider renaming `current_nav` to `original_nav` and retaining the old NAV in that field to clearly distinguish the request-time snapshot from the override value (`override_nav`).

Status: Resolved

26. `FUNDS_MANAGER_ADDRESS` is set unconditionally regardless of vault type

Severity: Informational

In `contracts/vault/src/contract/init.rs:32-33`, `FUNDS_MANAGER_ADDRESS` is saved during instantiation regardless of whether the vault is `InternallyManaged` or `ExternallyManaged`.

For `InternallyManaged` vaults, funds are held inside the contract and never forwarded to a funds manager, making the stored address unused and potentially confusing for developers or integrators inspecting the contract state.

Recommendation

We recommend either gating the `FUNDS_MANAGER_ADDRESS` storage behind a `VaultType::ExternallyManaged` check. Additionally, consider consolidating the funds manager into the `ROLES` storage to maintain a single source of truth for all privileged addresses.

Status: Resolved

27. Misleading doc comment describes unsupported behavior for `InternallyManaged`

Severity: Informational

In `contracts/vault/src/msg.rs:8`, the doc comment for `VaultType::InternallyManaged` states "*UpdateAum can only increase AUM; the updater must pay the difference.*"

This is misleading because the doc comment describes behavior that does not exist, as `execute_update_aum` rejects `InternallyManaged` vaults entirely in `contracts/vault/src/contract/exec.rs:438-439`.

Recommendation

We recommend updating the doc comment to accurately reflect that AUM changes for `InternallyManaged` vaults are made via `AddReward`.

Status: Resolved

28. Duplicate `CONTRACT_NAME` and `CONTRACT_VERSION` constants are defined

Severity: Informational

In `contracts/vault/src/lib.rs:38`, `CONTRACT_NAME` is defined as `"valdora-vault"`. The same constant with the same value is defined in `contracts/vault/src/contract.rs:14`.

This duplication creates a maintenance risk if one is updated without the other.

Recommendation

We recommend removing the unused `CONTRACT_NAME` and `CONTRACT_VERSION` declarations in `contracts/vault/src/lib.rs:38-40`.

Status: Resolved